

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

CO4 ASSESSMENT TOOL 1 – Test Case Design Assignment

Course Code:	CSA10	Course Title:	Software Engineering
CO Assessed:	CO4	Assessment:	Assessment Tool 1 – Test Case Design Assignment
Weightage:	20%	Total Marks:	25
CO4 Statement:	Implement robust testing, quality assurance, and DevOps practices, emphasizing security, reliability, and ethics		
Student Name:		Reg. No.:	
Date:		Duration:	60 minutes

Code of Conduct

I S. Jefna Princy - 192572058 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: _____

Annexure A – Test Case Design Assignment

Instructions to Students:

Each student is assigned an individual problem statement. Based on the assigned problem statement, **design and document comprehensive test cases** to verify the correctness, functionality, reliability, and usability of the proposed system.

Your test case design should include:

1. Identify the **functional and non-functional requirements** to be tested.
2. Identify the **test scenarios** for the assigned problem statement.
3. Prepare detailed **test cases** covering normal, boundary, invalid, and exceptional conditions.
4. Include **Test Case ID, Test Scenario, Test Steps, Test Data, Expected Result, Actual Result, and Pass/Fail Status**.
5. Apply appropriate **test design techniques** such as Equivalence Partitioning, Boundary Value Analysis, Decision Table, or State Transition Testing wherever applicable.
6. Ensure that the test cases provide adequate **requirement and functional coverage** for the assigned system.

Deliverable: Submit a Test Case Design document containing the identified scenarios, test cases, test data, expected results, and test coverage based on the individual problem statement.

Rubrics for Calculation

Evaluation Criteria	Excellent (4 Marks)	Good (3 Marks)	Satisfactory (2 Marks)	Needs Improvement (1 Mark)
---------------------	---------------------	----------------	------------------------	----------------------------

Requirement & Test Scenario Identification(15%)	Clearly identifies all relevant functional/non-functional requirements and comprehensive test scenarios	Identifies most requirements and scenarios	Identifies basic requirements and scenarios	Requirements/scenarios are incomplete or unclear
Test Case Design & Completeness(25%)	Comprehensive test cases covering normal, boundary, invalid, and exceptional conditions	Covers most important conditions with minor gaps	Covers basic conditions but misses several important cases	Test cases are very limited or incomplete
Test Data & Expected Results(20%)	Appropriate test data with precise, measurable expected results for every test case	Mostly appropriate test data and expected results	Some test data/results are unclear or incomplete	Test data and expected results are largely missing/incorrect
Application of Test Design Techniques(15%)	Correctly applies multiple techniques such as Equivalence Partitioning, Boundary Value Analysis, Decision Table, and State Transition	Correctly applies at least two relevant techniques	Applies one technique with limited justification	Techniques are not applied or incorrectly applied
Traceability & Test Coverage(15%)	Excellent mapping between requirements, scenarios, and test cases with strong coverage	Good traceability and coverage with minor gaps	Partial traceability and moderate coverage	Poor/no traceability and inadequate coverage
Documentation & Presentation(10%)	Well-structured, clear, professional, consistent, and easy to understand	Well-organized with minor presentation issues	Adequately organized but has some clarity/formatting issues	Poorly organized, unclear, or incomplete documentation

Criteria	Mark
Requirement & Test Scenario Identification (15%)	/5
Test Case Design & Completeness (25%)	/4
Test Data & Expected Results (20%)	/4
Application of Test Design Techniques (15%)	/4
Traceability & Test Coverage (15%)	/4
Documentation & Presentation (10%)	/4
TOTAL	/25

Course code: CSA1013

Name: S Jefna Princy

Course: Software Engineering

Reg No.: 192572058

TEST CASE DESIGN ASSIGNMENT

AI-Driven Smart Urban Noise Pollution Monitoring and Mitigation System

1. Introduction

The proposed system uses **IoT sensors, AI, GIS and cloud services** to continuously monitor urban noise levels. It collects noise data, identifies noise hotspots, predicts future noise levels and generates alerts when permitted noise limits are exceeded.

The test case design verifies whether the system works correctly under **normal, boundary, invalid and exceptional conditions**.

2. Functional Requirements to be Tested

ID	Functional Requirement
FR01	The system shall collect noise data from IoT sensors.
FR02	The system shall display real-time noise levels on the dashboard.
FR03	The system shall identify high-noise areas using GIS.
FR04	The system shall predict future noise levels using AI.
FR05	The system shall generate alerts when noise exceeds the permitted limit.
FR06	The system shall identify major sources of noise pollution.
FR07	The system shall generate environmental compliance reports.
FR08	The system shall support authorized users in viewing monitoring information.
FR09	The system shall store collected noise data for analysis.
FR10	The system shall continue monitoring when temporary sensor/network failures occur.

3. Non-Functional Requirements to be Tested

ID	Requirement	Test Condition
NFR01	Performance	Noise data should be processed with minimum delay.
NFR02	Scalability	System should support increasing sensors and users.
NFR03	Security	Unauthorized users must not access protected data.
NFR04	Reliability	Temporary failures should not stop the complete system.
NFR05	Availability	Monitoring service should remain continuously available.
NFR06	Maintainability	Modules should be easy to update.
NFR07	Usability	Dashboard should be simple and easy to understand.

System architecture of the proposed noise monitoring system

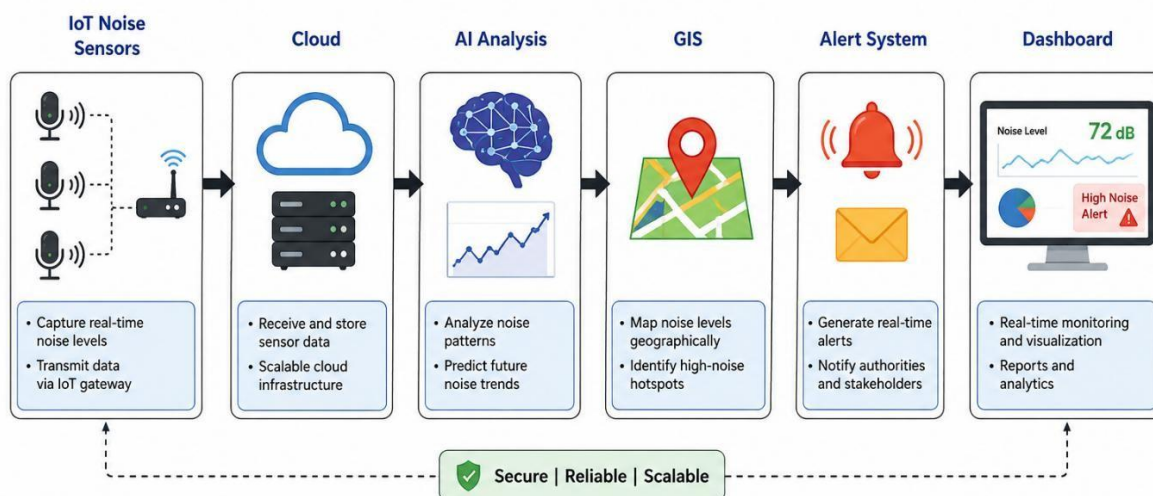


Figure 2: System architecture of the proposed noise monitoring system

4. Test Scenarios

These are the main things we are going to test:

1. Check whether sensors send noise data correctly.
2. Check whether real-time noise levels are displayed.
3. Check noise values at normal levels.
4. Check noise values at the permitted limit.
5. Check noise values above the permitted limit.
6. Check invalid sensor data.
7. Check missing sensor data.
8. Check GIS hotspot identification.
9. Check AI noise prediction.
10. Check excessive-noise alerts.
11. Check identification of noise sources.
12. Check environmental report generation.
13. Check unauthorized access.
14. Check system response during network failure.
15. Check dashboard usability.
16. Check system performance with many sensor readings.

6. Boundary Value Analysis

For the noise threshold, assume the configured permitted limit is **75 dB** for testing. We test values around the boundary:

Test	Noise Level	Expected Result
Below boundary	74 dB	Accepted; no excessive-noise alert
Boundary	75 dB	Correctly handled according to configured threshold
Above boundary	76 dB	Excessive-noise alert generated

This tests whether the system correctly handles values **just below, at and just above the limit**.

7. Equivalence Partitioning

We divide noise input into groups where the system should behave similarly.

Partition	Example Input	Expected
Valid normal value	50 dB	Accept
Valid high value	90 dB	Accept + generate alert
Boundary value	75 dB	Handle according to threshold
Negative value	-10 dB	Reject
Non-numeric value	ABC	Reject
Empty value	Blank	Reject

This reduces the number of tests while still covering different types of input.

8. Decision Table Testing

For noise alerts:

Condition	Case 1	Case 2	Case 3	Case 4
Noise below limit	Yes	No	Yes	No
Noise above limit	No	Yes	No	Yes
Sensor available	Yes	Yes	No	No
Generate alert	No	Yes	No	No
Display sensor error	No	No	Yes	Yes

Example:

If the **noise is above the limit AND sensor is available**, the system should **generate an alert**.

G. State Transition Testing

The monitoring system can have these states:

Monitoring → High Noise Detected → Alert Generated → Alert Acknowledged → Monitoring

Another possible state:

Monitoring → Sensor Failure → Error/Offline → Sensor Reconnected → Monitoring

This verifies that the system moves correctly from one state to another.

10. Non-Functional Test Cases

TC ID	Requirement	Test	Expected Result
NFT01	Performance	Send continuous sensor readings	Data should be processed with minimum delay
NFT02	Scalability	Increase number of sensors	System should continue operating without major performance degradation
NFT03	Security	Attempt unauthorized login	Access should be denied
NFT04	Reliability	Temporarily disconnect a sensor	System should continue monitoring available sensors
NFT05	Availability	Access dashboard during continuous monitoring	Dashboard should remain available
NFT06	Usability	Ask a user to locate noise level and alerts	User should understand the dashboard easily
NFT07	Maintainability	Update one software module	Other modules should continue functioning

11. Requirement Traceability Matrix

Requirement	Test Cases	Coverage
FR01 – Sensor data collection	TC01, TC07	Covered
FR02 – Real-time dashboard	TC02, TC15	Covered
FR03 – GIS hotspot detection	TC08	Covered

Requirement	Test Cases	Coverage
FR04 – AI prediction	TC09	Covered
FR05 – Noise alerts	TC04, TC05, TC10	Covered
FR06 – Noise source analysis	TC11	Covered
FR07 – Compliance reports	TC12	Covered
FR08 – Authorized access	TC13	Covered
FR09 – Data storage/analysis	TC01, TC09, TC12	Covered
FR10 – Failure handling	TC07, TC14	Covered
NFR01 – Performance	NFT01	Covered
NFR02 – Scalability	NFT02	Covered
NFR03 – Security	NFT03	Covered
NFR04 – Reliability	NFT04	Covered
NFR05 – Availability	NFT05	Covered
NFR06 – Maintainability	NFT07	Covered
NFR07 – Usability	NFT06	Covered

Requirement Coverage

All identified functional and non-functional requirements are mapped to at least one test case. Therefore, the test design provides **complete requirement traceability**.

12. Test Coverage Summary

Category	Number
Functional test cases	16
Non-functional test cases	7
Total test cases	23
Functional requirements covered	10/10
Non-functional requirements covered	7/7
Test techniques applied	4

13. Test Result Format

When you actually execute the test cases, update the last two columns. For example:

TC ID	Expected Result	Actual Result	Status
-------	-----------------	---------------	--------

TC05	Alert generated for 90 dB	Alert displayed correctly	PASS
------	---------------------------	---------------------------	------

If something doesn't work:

TC ID	Expected Result	Actual Result	Status
-------	-----------------	---------------	--------

TC05	Alert generated	No alert displayed	FAIL
------	-----------------	--------------------	------

Don't simply mark everything PASS unless you actually test it.